

CS404HW4 - Yusuf Eralp Kızıloğlu 29204

q1-)

1. hala(X,Y) - Paternal aunt. X is the hala of Y when X is Y's father's sister.

hala(X,Y) :-
 father(F,Y),
 sister(X,F).

2. teyze(X,Y) - Maternal aunt. X is the teyze of Y when X is Y's mother's sister.

teyze(X,Y) :-
 mother(M,Y),
 sister(X,M).

3. dayi(X,Y) - Maternal uncle. X is the dayi of Y when X is Y's mother's brother.

dayi(X,Y) :-
 mother(M,Y),
 brother(X,M).

4. amca(X,Y) - Paternal uncle. X is the amca of Y when X is Y's father's brother.

amca(X,Y) :-
 father(F,Y),
 brother(X,F).

5. anneanne(X,Y) - Maternal grandmother. X is the anneanne of Y when X is the mother of Y's mother.

anneanne(X,Y) :-
 mother(M,Y),
 mother(X,M).

6. babaanne(X,Y) - Paternal grandmother. X is the babaanne of Y when X is the mother of Y's father.

babaanne(X,Y) :-
 father(F,Y),
 mother(X,F).

7. gelin(X,Y) - Daughter-in-law. X is the gelin of Y when X is a female married to Y's son.

gelin(X,Y) :-
 son(S,Y),
 spouse(X,S),
 female(X).

8. damat(X,Y) - Son-in-law. X is the damat of Y when X is a male married to Y's daughter.

damat(X,Y) :-
 daughter(D,Y),
 spouse(X,D),
 male(X).

9. torun(X,Y) - Grandchild. X is the torun of Y when X is a grandchild of Y.

torun(X,Y) :-
 grandchild(X,Y).

10. dunur(X,Y) - Parents whose children are married to each other.
X and Y are dunur when a child of X is married to a child of Y.

dunur(X,Y) :-
 parent(X,C1),
 parent(Y,C2),
 spouse(C1,C2),
 X \= Y.

q2-)

The predicate I wrote uses the @< operator to ensure alphabetical ordering,
so each couple only appears once. The fail at the end forces backtracking to find all solutions, and the second clause succeeds after all pairs are printed.

Query: list_all_married_couples.

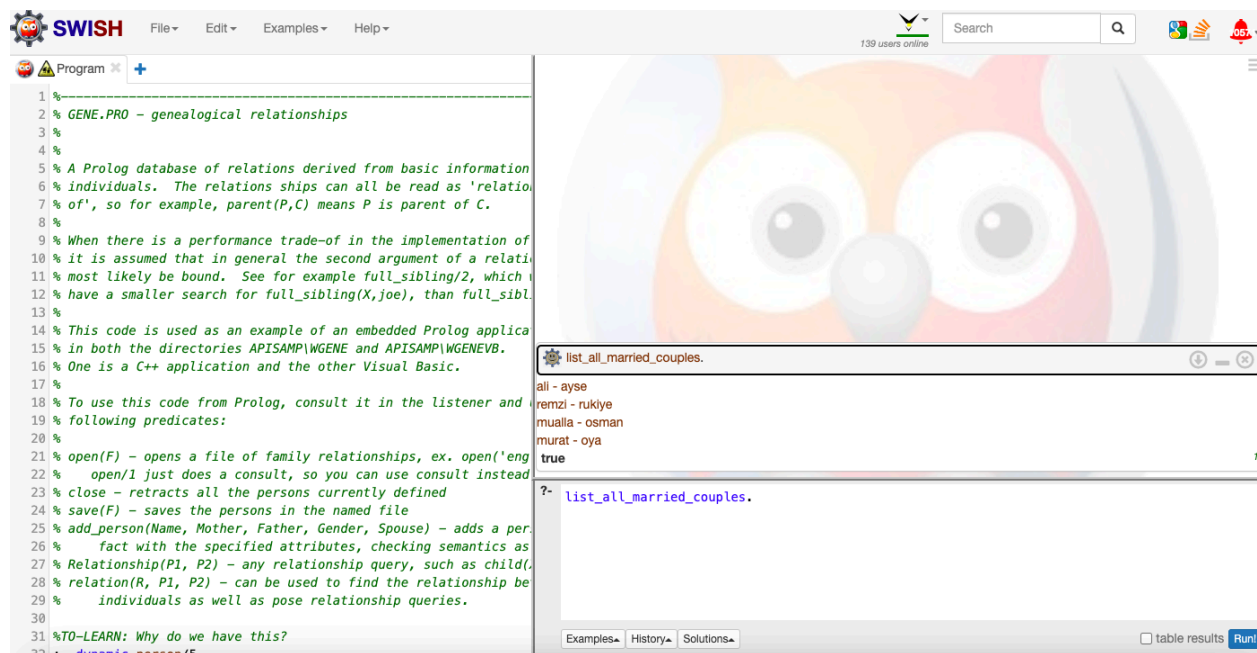
ali - ayse

remzi - rukiye

mualla - osman

murat - oya

true.



The screenshot shows the SWISH Prolog IDE interface. The left pane displays a Prolog program with comments and predicates. The right pane shows the execution results for the query `list_all_married_couples.`.

```
1 %  
2 % GENE.PRO - genealogical relationships  
3 %  
4 %  
5 % A Prolog database of relations derived from basic information  
6 % individuals. The relations ships can all be read as 'relation  
7 % of', so for example, parent(P,C) means P is parent of C.  
8 %  
9 % When there is a performance trade-of in the implementation of  
10 % it is assumed that in general the second argument of a relation  
11 % most likely be bound. See for example full_sibling/2, which  
12 % have a smaller search for full_sibling(X,joe), than full_sibl  
13 %  
14 % This code is used as an example of an embedded Prolog applica  
15 % in both the directories APISAMP\WGENE and APISAMP\WGENEVB.  
16 % One is a C++ application and the other Visual Basic.  
17 %  
18 % To use this code from Prolog, consult it in the listener and  
19 % following predicates:  
20 %  
21 % open(F) - opens a file of family relationships, ex. open('eng  
22 % open/1 just does a consult, so you can use consult instead  
23 % close - retracts all the persons currently defined  
24 % save(F) - saves the persons in the named file  
25 % add_person(Name, Mother, Father, Gender, Spouse) - adds a per  
26 % fact with the specified attributes, checking semantics as  
27 % Relationship(P1, P2) - any relationship query, such as child(  
28 % relation(R, P1, P2) - can be used to find the relationship be  
29 % individuals as well as pose relationship queries.  
30 %  
31 %TO-LEARN: Why do we have this?  
32 :- dynamic person/5.
```

Execution results for `list_all_married_couples.`:

```
ali - ayse  
remzi - rukiye  
mualla - osman  
murat - oya  
true
```

The bottom of the interface shows tabs for Examples, History, and Solutions, along with a checkbox for "table results" and a "Run" button.

q-3)

Yes, there are people who have at least two married children.

These are: Ayse, Ali, Rukiye, and Remzi. Each of them has two children who are married to someone.

parent(P,C1), parent(P,C2), C1 \= C2, spouse(_,C1), spouse(_,C2).

The screenshot shows the SWISH Prolog environment. The left pane contains a Prolog program with the following code:

```

% A Prolog database of relations derived from basic information
% individuals. The relations ships can all be read as 'relation
% of', so for example, parent(P,C) means P is parent of C.
%
% When there is a performance trade-of in the implementation of
% it is assumed that in general the second argument of a relation
% most likely be bound. See for example full_sibling/2, which
% have a smaller search for full_sibling(X,joe), than full_sibling(X,joe).
%
% This code is used as an example of an embedded Prolog application
% in both the directories APISAMP\WGENE and APISAMP\WGENEV.B.
% One is a C++ application and the other Visual Basic.
%
% To use this code from Prolog, consult it in the listener and
% following predicates:
%
% open(F) - opens a file of family relationships, ex. open('eng
% open/1 just does a consult, so you can use consult instead
% close - retracts all the persons currently defined
% save(F) - saves the persons in the named file
% add_person(Name, Mother, Father, Gender, Spouse) - adds a person
% fact with the specified attributes, checking semantics as
% Relationship(P1, P2) - any relationship query, such as child,
% relation(R, P1, P2) - can be used to find the relationship between
% individuals as well as pose relationship queries.
%
%T0-LEARN: Why do we have this?
% :- dynamic person/5.
%
parent(P,C) :-
    (mother(P,C) ; father(P,C)).

```

The right pane shows the execution results for the query `parent(P,C1), parent(P,C2), C1 \= C2, spouse(_,C1), spouse(_,C2).`. The results are:

```

C1 = ayse,
C2 = all,
P = unknown
C1 = ayse,
C2 = remzi,
P = unknown
C1 = ayse,
C2 = rukiye,
P = unknown
C1 = ayse,
C2 = all,
P = unknown
C1 = ayse,
C2 = remzi,
P = unknown
C1 = ayse,
C2 = rukiye,
P = unknown
C1 = all,
C2 = ayse,
P = unknown
C1 = all,
C2 = remzi,
P = unknown

```

At the bottom, there is a button labeled "table results" and a "Run" button.

q-4)

YES, the order matters.

The `add()` call must come before `ancestor_check()`, `mother_check()`, and `father_check()` because these validation predicates need to query the

database to check for logical inconsistencies. If we move `add()` to the end, the person would not be in the database yet when the checks run, so the `ancestor_check` would never detect if someone is their own ancestor, and the mother/father checks would not work properly either.

q-5)

The `mother_check` predicate validates the mother information when adding a new person. It has four clauses that work as follows:

First clause: `mother_check(_, _, Mother) :- \+(person(Mother)), !.`

It first checks whether the Mother is NOT already a person in the database. If this succeeds (meaning the mother is unknown or not yet added), it cuts and succeeds immediately. This allows us to add people whose mothers are not in the database without triggering any errors.

Second clause: `mother_check(_, _, Mother) :- male(Mother), ...`

If the first clause failed (mother IS in database), it checks if Mother is male. If this succeeds, it means we are trying to assign a male person as someone's mother, which is wrong. It asserts an error message and fails.

Third clause: `mother_check(Name, male, _) :- mother(Name, _), ...`

This checks if the new person being added (who is male based on the second argument) is already recorded as the mother of someone else in the database. If this succeeds, it means a male person is listed as a mother, which is a logical error. It asserts an error message and fails.

Fourth clause: `mother_check(_,_,_).`

This is the catch-all clause that simply succeeds. If none of the error

conditions above were triggered, the mother information is considered valid.

In summary, mother_check ensures that: (1) if a mother is specified and exists in the database, she must be female, and (2) if the person being added is male, they should not already be someone's mother.